Journal of
Risk and Financial Management

MDPI

Article

Does CNN-Based Feature Extraction Improve High-Frequency Return Prediction? Evidence from the CSI 300 Index

Fan Zhang [1,*] and Haobing Wang [2]

[1] Independent Researcher, Chengdu 610095, China
[2] Department of Econometrics and Business Statistics, Faculty of Business and Economics, Monash University, Clayton, VIC 3800, Australia; hwan0332@student.monash.edu
* Correspondence: fan5211v@163.com

Abstract

This study investigates whether CNN-based front-end feature extraction improves the predictive performance of deep learning models applied to 1 min intraday CSI 300 index data. Three baseline sequence models, LSTM, GRU, and TCN, are compared against their CNN hybrid and dual-branch fusion variants across five input window sizes, with all comparisons using identical back-end configurations. A total of 45 model configurations are trained and evaluated across 20 independent runs, with performance assessed on four metrics (MAE, RMSE, Directional Accuracy, and Information Coefficient) and statistical significance evaluated by paired t-tests. After standardisation, adding a CNN front-end does not consistently improve performance over the raw baseline and reduces IC for LSTM- and GRU-based models in many cases (e.g., IC of 0.0187 vs. 0.1031 for CNN-LSTM vs. LSTM at $W = 1$), suggesting that standardised recurrent models can extract useful patterns directly from the raw sequence without CNN preprocessing. The dual-branch fusion architecture, which retains both the raw and CNN-compressed sequence branches, consistently outperforms the pure CNN hybrid on MAE, RMSE, and IC for LSTM- and GRU-based models (e.g., LSTMDualBranchFusion achieves statistically significant MAE reductions over CNN-LSTM at $W = 1$, $W = 2$, $W = 4$, and $W = 5$), indicating that the raw sequence carries complementary predictive information that the CNN front-end discards. TCN-based models produce near-zero or negative IC values regardless of architecture variant, suggesting a possible limitation of dilated convolutional architectures for return rank-ordering on this dataset and sample period. These findings are consistent across all five window sizes examined.

Keywords: high-frequency financial forecasting; convolutional neural network (CNN); feature extraction; feature standardisation; long short-term memory (LSTM); gated recurrent unit (GRU); temporal convolutional network (TCN); dual-branch fusion; CSI 300 Index

Check for updates

Academic Editor: Ruipeng Liu

Received: 23 April 2026
Revised: 15 May 2026
Accepted: 18 May 2026
Published: 20 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

The increasing availability of ultra-high-frequency financial data has created new opportunities for more informative return prediction using deep learning models. However, raw high-frequency data present significant challenges for sequential modelling: as sampling frequency increases, market microstructure noise becomes increasingly dominant relative to the underlying price signal (Ait-Sahalia et al., 2005), and very long input sequences impose substantial computational and optimisation burdens on deep learning

architectures (Luo & Tian, 2020). These challenges motivate the exploration of preprocessing strategies that can reduce noise and compress the input sequence before it is passed to a downstream predictive model.

Convolutional neural networks (CNNs) have emerged as a natural candidate for this role. When applied as a front-end feature extractor with a sliding window, CNN layers can aggregate local temporal patterns across multiple time steps, effectively downsampling the input sequence while learning compact representations (Livieris et al., 2020). A natural extension of this hybrid design is the dual-branch fusion architecture, in which CNN-extracted features and raw sequence features are processed through parallel branches and subsequently fused (Kim & Kim, 2019).

Despite the widespread adoption of these architectures, there is a lack of systematic empirical evidence on whether CNN-based feature extraction consistently improves predictive performance over direct sequence modelling on ultra-high-frequency intraday data, and whether dual-branch fusion provides additional gains beyond a pure CNN hybrid design. Most existing studies evaluate a single hybrid architecture against a single baseline without systematically controlling for model family, input horizon, or random initialisation variability (Henderson et al., 2018), making it difficult to draw robust conclusions about the relative merits of these design choices.

This study addresses these gaps by examining two research questions: (RQ1) does CNN-based front-end feature extraction improve predictive performance over direct sequence modelling on standardised 1 min data, and (RQ2) does dual-branch fusion provide additional gains beyond a pure CNN hybrid design? To address these questions, we conduct a systematic comparative evaluation of three model families: LSTM (Hochreiter & Schmidhuber, 1997), GRU (Cho et al., 2014), and TCN (Bai et al., 2018), alongside their CNN hybrid and dual-branch fusion variants, applied to 1 min CSI 300 index data spanning January 2015 to December 2025 (C. Li & Qian, 2022; Song et al., 2024). All CNN hybrid and dual-branch fusion models use identical back-end configurations to their raw baseline counterparts, ensuring that any performance differences are attributable solely to the presence or absence of the CNN front-end or the additional raw sequence branch. Five input window sizes ranging from 1 to 5 trading days are examined, and all models are trained and evaluated across 20 independent runs to account for initialisation variability, with statistical significance assessed via paired t-tests (Dietterich, 1998).

The main contributions of this study are threefold. First, we show that, after standardisation, adding a CNN front-end does not consistently improve predictive performance over the raw baseline, and in many cases reduces IC for LSTM- and GRU-based models, suggesting that standardised recurrent models are capable of extracting useful patterns directly from the raw 1 min sequence without CNN preprocessing. Second, we show that the dual-branch fusion architecture, which retains both the raw and CNN-compressed sequence branches, consistently outperforms the pure CNN hybrid on MAE, RMSE, and IC for LSTM- and GRU-based models, indicating that the raw sequence carries complementary predictive information that the CNN front-end discards. Third, we find that TCN-based models produce near-zero or negative IC values regardless of architecture variant or window size, suggesting a possible limitation of dilated convolutional architectures for return rank-ordering on this dataset and sample period that is not resolved by CNN preprocessing or dual-branch fusion.

The remainder of this paper is organised as follows. Section 2 reviews related work on high-frequency data noise, deep learning for financial forecasting, and CNN-based feature extraction. Section 3 describes the data, features, model architectures, and experimental setup. Section 4 presents the empirical results and Section 5 discusses their implications. Section 6 concludes.

2. Literature Review

2.1. High-Frequency Data Noise and Sampling Frequency

High-frequency financial data has become increasingly available in recent years, but more frequent observations do not necessarily yield more predictive information. Ait-Sahalia et al. (2005) show that as sampling frequency increases, market microstructure noise becomes increasingly dominant relative to the underlying efficient price movement, leading to a deterioration in the signal-to-noise ratio of observed prices. Subsequent research by Aït-Sahalia and Yu (2008) emphasizes that microstructure noise is not purely statistical in nature, but originates from underlying trading frictions, including bid–ask bounce, price discreteness, and order-processing effects.

To address this issue, some studies suggest that sampling at lower frequencies can help improve the signal-to-noise ratio in observed returns. In particular, Bandi and Russell (2006) show that while ultra-high-frequency data are heavily contaminated by microstructure noise, lower-frequency returns are more informative about the underlying efficient price process. Furthermore, Liu et al. (2015) provide empirical evidence that sampling at 5 min intervals leads to more reliable signal extraction than ultra-high-frequency observations.

In addition, some studies have explored combining information across sampling frequencies in the presence of market microstructure noise. Rather than relying solely on either ultra-high-frequency or sparsely sampled returns, this line of work aims to balance the informational gains from dense intraday observations against the noise reduction achieved through lower-frequency aggregation. For example, Andersen et al. (2011) consider estimators such as the average estimator, which aggregates information across multiple sparse sampling grids and helps reduce noise while retaining useful intraday variation.

Taken together, these findings motivate the exploration of learned feature extraction methods, such as convolutional neural networks, which can aggregate local temporal patterns across a sliding window before passing compressed representations to downstream predictive models.

2.2. Deep Learning with High-Frequency Financial Data

High-frequency financial data have increasingly been used in conjunction with deep learning models for short-horizon return prediction and market dynamics modelling across a wide range of financial instruments. Among the most commonly adopted architectures are recurrent neural networks, particularly Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models, as well as convolution-based sequence models such as Temporal Convolutional Networks (TCNs), all of which have been widely applied to sequential financial data.

Among deep learning methods for financial forecasting, recurrent architectures such as LSTM and GRU have been widely regarded as standard choices for sequential modelling, including in high-frequency and intraday prediction tasks. For example, Luo and Tian (2020) proposed an LSTM-based framework for high-frequency financial forecasting on 1 min SSE 50 index data, where Sub-Step Grid Search (SGS) was employed to improve hyperparameter tuning efficiency. Their results showed that the proposed method outperformed traditional backpropagation (BP) neural networks under trending market conditions. At a lower intraday frequency, Song et al. (2024) introduced a hybrid nonparametric regression and LSTM (NR-LSTM) model for 5 min forecasting on the CSI 300 Index, Ping An Bank, the FTSE 100, and the S&P 500. Their findings suggest that decomposing the series into trend and residual components can improve prediction robustness under volatile and jump-prone market conditions. At a finer intraday frequency, C. Li and Qian (2022) proposed a frequency-decomposition-based GRU-transformer model on high-frequency limit order

book data from CSI-300 stocks, showing that decomposition-enhanced recurrent-attention architectures can improve high-frequency stock prediction.

Beyond recurrent architectures, Temporal Convolutional Networks (TCNs) have emerged as a compelling alternative for sequential modelling in financial applications. Dai et al. (2022) applied TCN to ultra-high-frequency transaction price data from constituent stocks of the Chinese Shenzhen Stock Exchange 100 Index (SZSE 100), modelling the discrete conditional distribution of tick-by-tick price changes across a dataset of nearly ten million transactions. Their results showed that TCN and its attention-augmented variant consistently outperformed both GARCH-family models and LSTMs in describing the dynamic process of the UHF price change sequence. At the lower end of the intraday frequency spectrum, C.-X. Zhang et al. (2022) demonstrated that TCN is effective for forecasting stock return volatility and value-at-risk, finding that TCN outperformed GRU and LSTM across multiple evaluation metrics, as well as conventional GARCH-type benchmarks.

While these architectures have shown strong performance in intraday forecasting, directly applying them to raw high-frequency data remains challenging, as recurrent models can struggle with very long input sequences and TCN remains sensitive to noise. This suggests that a dedicated feature extraction stage prior to sequential modelling may be beneficial.

2.3. CNN-Based Feature Extraction in Financial Time Series

Convolutional neural networks (CNNs) have demonstrated strong feature extraction capabilities when applied to financial time-series data. In the standalone setting, Gunduz et al. (2017) proposed a CNN architecture with a correlation-sorted tensor input to predict the intraday direction of Borsa Istanbul 100 stocks, showing that the spatial arrangement of features in the input matrix materially affected the quality of extracted representations. Hoseinzade and Haratizadeh (2019) proposed CNNpred, applying CNN to automatically extract features from a rich multivariate input of 82 variables across five major U.S. stock market indices, with convolutional layers designed to capture both cross-variable and cross-temporal patterns, significantly outperforming shallow ANN baselines.

In hybrid architectures, CNN layers are commonly used as a front-end feature extractor feeding into a recurrent module for downstream sequence modelling. Livieris et al. (2020) motivated this design choice by exploiting the ability of convolutional layers to extract useful knowledge and learn the internal representation of time-series data, before passing the extracted features to LSTM layers for gold price forecasting. Similarly, Z. Zhang et al. (2019) highlighted that the main motivation for incorporating CNN in their DeepLOB architecture was to automate feature extraction from high-frequency limit order book data, which is notoriously noisy and where hand-crafted features are difficult to design reliably.

Combining CNN-extracted representations with raw sequence features through dedicated parallel branches, dual-branch architectures have emerged as a natural extension beyond sequential feature extraction pipelines, fusing complementary information for downstream prediction. Kim and Kim (2019) demonstrated this principle in a feature fusion LSTM-CNN model that combined temporal features from LSTM and image-based features from CNN derived from the same underlying stock data, finding that the joint representation outperformed either branch in isolation. More recently, Singh and Raman (2026) proposed the dual-branch spectral-trend attention network (DB-STAN) for financial time series, explicitly routing heterogeneous signal components through dedicated convolutional encoders prior to fusion, achieving improved forecasting accuracy across multiple financial benchmarks. At the general time-series level, Stitsyuk and Choi (2025) proposed xPatch, a dual-stream architecture that concatenates outputs from a linear MLP stream

and a non-linear CNN stream, allowing the model to dynamically exploit both linear and non-linear temporal patterns.

Motivated by these findings, this study further investigates whether retaining the raw sequence branch alongside CNN-extracted features in a dual-branch fusion architecture can yield additional predictive gains over a pure CNN front-end design, with a specific focus on ultra-high-frequency 1 min intraday data from the CSI 300 index.

3. Materials and Methods

3.1. Data

This study uses 1 min intraday data for the CSI 300 index, spanning from January 2015 to December 2025. The CSI 300 index tracks the 300 largest and most liquid stocks listed on the Shanghai and Shenzhen stock exchanges, and has been widely used as the benchmark for the Chinese A-share market in prior empirical studies (C. Li & Qian, 2022; Song et al., 2024). The full sample is divided into three non-overlapping subsets following the standard walk-forward convention for financial time series (Cerqueira et al., 2020). A summary of the data split and dataset size for each split is provided in Table 1.

Table 1. Dataset summary.

Split	Date Range	Trading Days	Total 1 min Bars
Training	January 2015–December 2024	2430	583,200
Validation	January 2025–June 2025	117	28,080
Test	July 2025–December 2025	126	30,240

3.2. Feature and Label

For each 1 min bar, 18 features are constructed, comprising the raw OHLCV and amount fields, together with a set of candlestick-derived and return-based features, as summarised in Table 2.

Table 2. Feature definitions.

No.	Feature	Definition
1	$open_{D,M}$	Raw opening price
2	$high_{D,M}$	Raw high price
3	$low_{D,M}$	Raw low price
4	$close_{D,M}$	Raw closing price
5	$volume_{D,M}$	Raw trading volume
6	$amount_{D,M}$	Raw trading amount
7	$hl_mid_{D,M}$	$(high_{D,M} + low_{D,M})/2$
8	$oc_mid_{D,M}$	$(open_{D,M} + close_{D,M})/2$
9	$k_length_{D,M}$	$high_{D,M} - low_{D,M}$
10	$k_body_{D,M}$	$\lvert close_{D,M} - open_{D,M} \rvert$
11	$k_upper_shadow_{D,M}$	$high_{D,M} - \max(open_{D,M}, close_{D,M})$
12	$k_lower_shadow_{D,M}$	$\min(open_{D,M}, close_{D,M}) - low_{D,M}$
13	$k_body_direction_{D,M}$	$\text{sign}(close_{D,M} - open_{D,M}) \in \{-1, 0, 1\}$
14	$log_price_return_{D,M}$	$\log(close_{D,M}/close_{D,M-1})$
15	$volume_change_{D,M}$	$volume_{D,M} - volume_{D,M-1}$
16	$minute_of_day_norm_{D,M}$	$minute_of_day_{D,M}/240$
17	$is_day_open_{D,M}$	$\mathbf{1}[M \leq 5]$
18	$is_day_close_{D,M}$	$\mathbf{1}[M \geq 235]$

Note: D denotes the trading day and $M \in \{1, \ldots, 240\}$ denotes the 1 min bar index, where $M = 1$ is 09:31 and $M = 240$ is 15:00. $\mathbf{1}[\cdot]$ denotes the indicator function, equal to 1 if the condition is satisfied and 0 otherwise.

All features describe the price and volume properties of each individual 1 min bar. Multi-bar technical indicators such as moving averages are deliberately excluded, so that all temporal pattern extraction is left to the model rather than pre-built into the features (Z. Zhang et al., 2019). Log-transformed returns and volume changes are also used, as log returns have more stable statistical properties and are standard in the financial forecasting literature (Campbell et al., 1998). Intraday time features (*minute_of_day_norm*, *is_day_open*, *is_day_close*) are included to capture systematic intraday patterns such as the opening and closing effects, which have been documented in high-frequency equity markets (Admati & Pfleiderer, 1988).

Each trading day in the Chinese A-share market consists of 241 one-minute bars: 240 bars from the continuous trading session (09:31–15:00) and one bar from the opening call auction (09:30). Since *log_price_return* and *volume_change* require a one-period lag, the 09:30 bar produces a missing value and is dropped, leaving exactly 240 valid time steps per trading day.

All continuous features are standardised using z-score normalisation prior to model training, which is the most widely used normalisation scheme for deep learning applied to financial time series (Passalis et al., 2019):

$$\tilde{x} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{1}$$

where μ_{train} and σ_{train} denote the mean and standard deviation computed from the training set only. The same statistics are then applied to the validation and test sets to avoid data leakage (Cerqueira et al., 2020). Four features are excluded from standardisation: $k_body_direction_{D,M}$, $minute_of_day_norm_{D,M}$, $is_day_open_{D,M}$, and $is_day_close_{D,M}$. The first is a discrete ternary variable, and the latter three are already bounded within $[0, 1]$ by construction.

The prediction label is the forward log return $r_D = \log(open_{D+2}/open_{D+1})$, where $open_{D+1}$ and $open_{D+2}$ are the opening prices of the next two consecutive trading days after the observation day D. This label is adopted as a T+1 execution-protocol-oriented trading payoff target specific to the Chinese A-share market: a signal is formed using information available up to day D, a position is opened at the start of day $D + 1$, and the position is closed at the start of day $D + 2$, making this the earliest executable round-trip return under the T+1 rule. It should be noted that this differs from the conventional next-period return forecasting target commonly used in general return prediction studies; the reported MAE, RMSE, DA, and IC metrics therefore correspond to this specific execution protocol rather than to general next-period return prediction (Sun & Li, 2025).

3.3. Window Design

Five window sizes are considered, corresponding to 1 to 5 trading days of 1 min data. Given that each trading day contains 240 time steps, the input tensors have dimensions 240×18, 480×18, 720×18, 960×18, and 1200×18 for window sizes of 1 through 5 days, respectively. Examining a range of input horizons is a common practice in financial forecasting studies to assess whether model performance is sensitive to the choice of lookback window (Luo & Tian, 2020; Song et al., 2024).

For models with a CNN front-end, the convolutional layer uses a kernel size of 5 and stride of 5, so each output step aggregates exactly 5 consecutive 1 min bars, effectively downsampling the sequence from 1 min to 5 min frequency. This design is motivated by the finding of Liu et al. (2015) that 5 min sampling improves signal quality in high-frequency data. For baseline models without a CNN front-end, the raw 1 min sequence is passed directly to the downstream model.

3.4. Models

3.4.1. Baseline Models

Three baseline models are employed: LSTM, GRU, and TCN. All three models receive the raw 1 min input sequence of shape $T \times 18$ directly, where $T \in \{240, 480, 720, 960, 1200\}$ depending on the window size.

LSTM

The Long Short-Term Memory (LSTM) network (Hochreiter & Schmidhuber, 1997) is a recurrent architecture that addresses the vanishing gradient problem through a forget gate, an input gate, and an output gate, which together control how information is stored and discarded across time steps. LSTM has been widely applied to financial time-series forecasting, including high-frequency intraday prediction tasks (Luo & Tian, 2020; Song et al., 2024).

In this study, the LSTM model comprises 2 stacked unidirectional layers with a hidden size of 64 and a dropout rate of 0.2 applied between layers. The hidden state at the final time step $\mathbf{h}_T$ is passed through a prediction head consisting of a linear layer projecting to 16 units with ReLU activation, followed by a linear layer producing the scalar output $\hat{r}_t$. The overall architecture is illustrated in Figure 1.

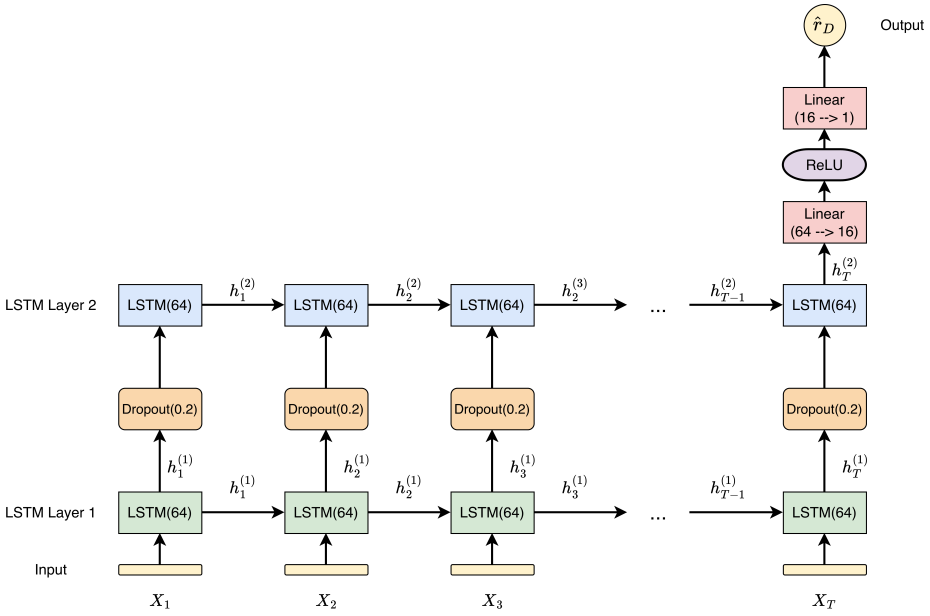


Figure 1. Overall architecture of LSTM. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T \times 18}$ of day D, where $X_t \in \mathbb{R}^{1 \times 18}$ denotes the feature vector at each time step t, and outputs the predicted return $\hat{r}_D$ for day D.

GRU

The Gated Recurrent Unit (GRU) (Cho et al., 2014) is a simplified variant of LSTM that merges the forget and input gates into a single update gate, reducing the number of parameters while retaining the ability to model long-range dependencies. GRU has similarly been used in financial forecasting applications (C. Li & Qian, 2022; C.-X. Zhang et al., 2022).

In this study, the GRU model comprises 2 stacked unidirectional layers with a hidden size of 64 and a dropout rate of 0.2 applied between layers. The hidden state at the final time step $\mathbf{h}_T$ is passed through the same prediction head as LSTM, producing the scalar output $\hat{r}_t$. The overall architecture is illustrated in Figure 2.

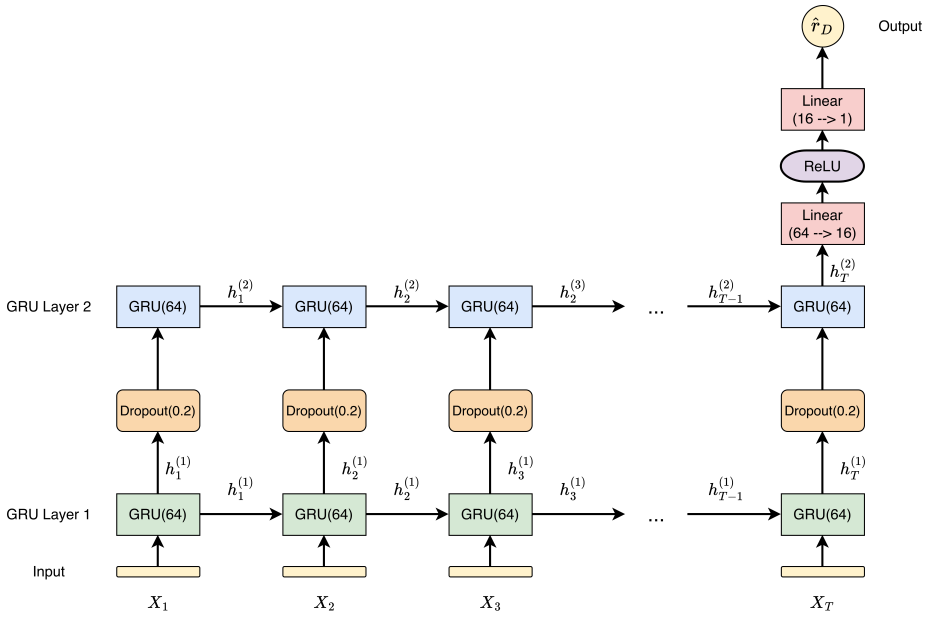


Figure 2. Overall architecture of GRU. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, where $X_t \in \mathbb{R}^{1\times 18}$ denotes the feature vector at each time step t, and outputs the predicted return $\hat{r}_D$ for day D.

TCN

The Temporal Convolutional Network (TCN) (Bai et al., 2018) is a convolutional architecture for sequence modelling that uses causal convolutions, ensuring the output at time t depends only on past inputs, and dilated convolutions, which expand the receptive field exponentially without adding parameters. TCN has shown competitive performance in financial time-series tasks (Dai et al., 2022; C.-X. Zhang et al., 2022). A dilated causal convolution with dilation factor d and kernel size k is defined as:

$$(\mathbf{x} *_d \mathbf{f})(t) = \sum_{i=0}^{k-1} \mathbf{f}(i) \cdot \mathbf{x}(t - d \cdot i) \tag{2}$$

where $*_d$ denotes the dilated convolution operator with dilation factor d, and $\mathbf{f}$ denotes the convolutional filter. Figure 3 illustrates the dilated causal convolution mechanism with dilation factors $d \in \{1, 2, 4\}$ for clarity. The full model uses five layers with $d \in \{1, 2, 4, 8, 16\}$.

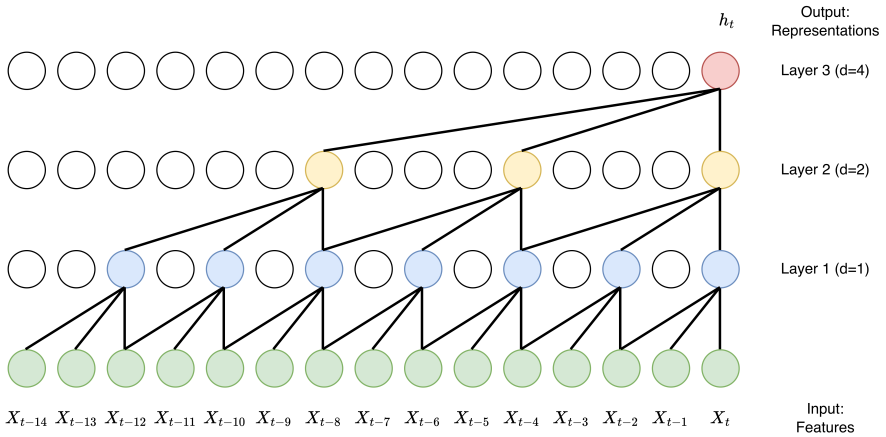


Figure 3. Dilated causal convolution mechanism of TCN. Each $X_t \in \mathbb{R}^{1\times 18}$ denotes the feature vector at intraday time step t within the input window used to predict the daily return. Colours indicate receptive field contributions: green = input layer, blue = Layer 1 ($d = 1$), yellow = Layer 2 ($d = 2$), red = output node at Layer 3 ($d = 4$).

Each TCN block consists of two such dilated causal convolutions with a residual connection:

$$\mathbf{h}^{(l)} = \mathrm{ReLU}\Big(\mathcal{F}(\mathbf{h}^{(l-1)}) + \mathbf{h}^{(l-1)}\Big) \tag{3}$$

where $\mathcal{F}(\cdot)$ denotes the two-layer dilated causal convolution transformation and $\mathbf{h}^{(l)}$ denotes the output of the l-th block.

In this study, the TCN model uses 5 blocks with dilation factors $\{1, 2, 4, 8, 16\}$, a kernel size of 3, an output channel size of 64, and a dropout rate of 0.2. The temporal dimension is aggregated via global average pooling, and the resulting representation is passed through a prediction head identical to LSTM and GRU, producing the scalar output $\hat{r}_t$. The overall architecture is illustrated in Figure 4.

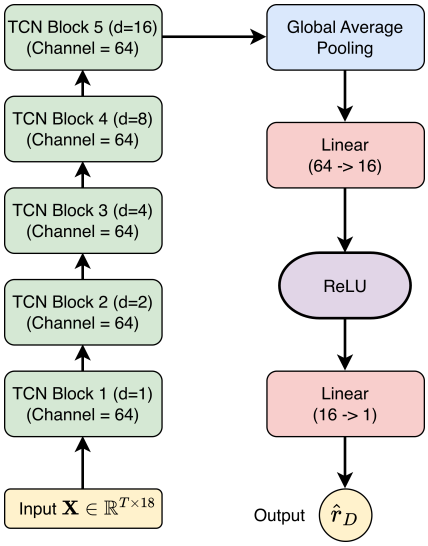


Figure 4. Overall structure of TCN. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

3.4.2. CNN Hybrid Models

The CNN hybrid models extend the baseline architectures by adding a 1D convolutional front-end before the downstream sequence model. The CNN front-end serves two purposes: first, it aggregates local temporal patterns across a sliding window, compressing the raw 1 min sequence into a lower-frequency representation with reduced microstructure noise (Ait-Sahalia et al., 2005); second, the compressed sequence is shorter, which reduces the computational burden on the downstream model when processing long input sequences. This design follows the approach of Livieris et al. (2020) and Z. Zhang et al. (2019).

Formally, given an input sequence $\mathbf{X} \in \mathbb{R}^{T\times 18}$, the CNN front-end applies a 1D convolution with kernel size $k = 5$ and stride $s = 5$, producing a compressed representation $\mathbf{Z} \in \mathbb{R}^{\lfloor T/5 \rfloor \times 64}$:

$$\mathbf{Z} = \mathrm{ReLU}(\mathrm{Conv1D}(\mathbf{X}; k = 5, s = 5, C_{\text{out}} = 64)) \tag{4}$$

The stride of 5 ensures that each output step aggregates exactly 5 consecutive 1 min bars, effectively downsampling the input from 1 min to 5 min frequency, consistent with the finding of Liu et al. (2015). The compressed sequence $\mathbf{Z}$ is then passed to the downstream model (LSTM, GRU, or TCN), following the same architecture and hyperparameter settings as described in Section 3.4.1, yielding three hybrid models: CNN-LSTM (Figure 5), CNN-GRU (Figure 6), and CNN-TCN (Figure 7).

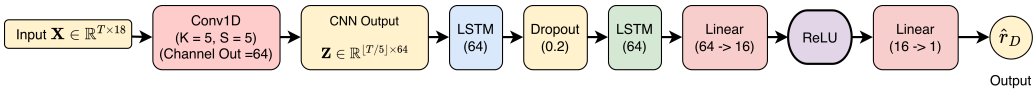


Figure 5. Overall structure of CNN-LSTM. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

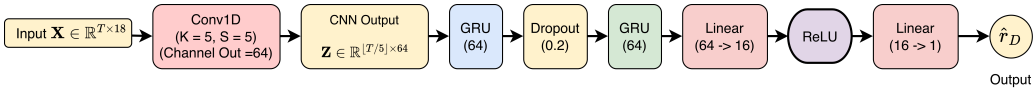


Figure 6. Overall structure of CNN-GRU. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

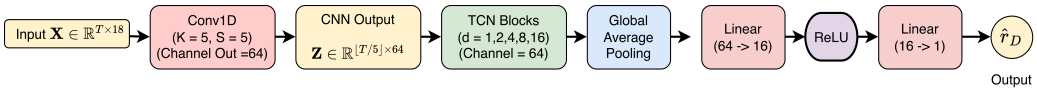


Figure 7. Overall structure of CNN-TCN. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

3.4.3. Dual-Branch Fusion Models

The dual-branch fusion models investigate whether retaining the raw sequence alongside CNN-extracted features provides additional predictive gains over the pure CNN hybrid design. This architecture is motivated by prior work showing that combining CNN-extracted and raw sequence representations through parallel branches can improve forecasting performance (Kim & Kim, 2019; Singh & Raman, 2026).

Each dual-branch model takes the same input $\mathbf{X} \in \mathbb{R}^{T\times 18}$ and processes it through two parallel branches: a raw branch, which is the corresponding baseline model (LSTM, GRU, or TCN) processing the input sequence directly, and a CNN branch, which is the corresponding CNN hybrid model processing the CNN-compressed sequence. The encoded representations from both branches are concatenated to form a joint representation:

$$\mathbf{z}_{\text{fusion}} = \text{Concat}(\mathbf{z}_{\text{raw}}, \mathbf{z}_{\text{cnn}}) \in \mathbb{R}^{128} \tag{5}$$

where $\mathbf{z}_{\text{raw}} \in \mathbb{R}^{64}$ and $\mathbf{z}_{\text{cnn}} \in \mathbb{R}^{64}$ denote the encoded representations from the raw and CNN branches respectively. The fused representation $\mathbf{z}_{\text{fusion}}$ is then passed through a fusion head consisting of a linear layer projecting to 64 units with ReLU activation and dropout of 0.2, followed by a linear layer producing the scalar output $\hat{r}_t$. This yields three dual-branch fusion models: LSTMDualBranchFusion (Figure 8), GRUDualBranchFusion (Figure 9), and TCNDualBranchFusion (Figure 10).

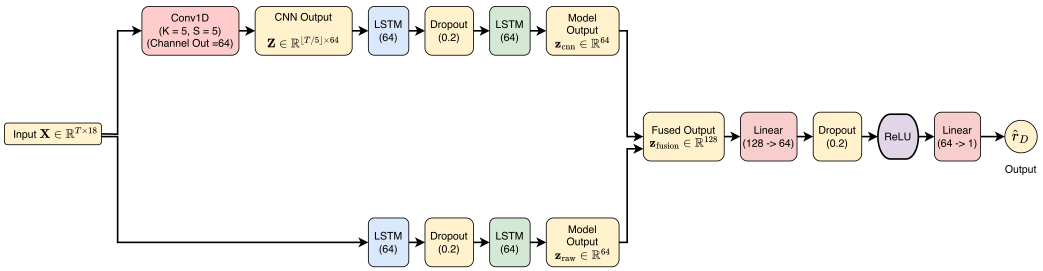


Figure 8. Overall structure of LSTMDualBranchFusion. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

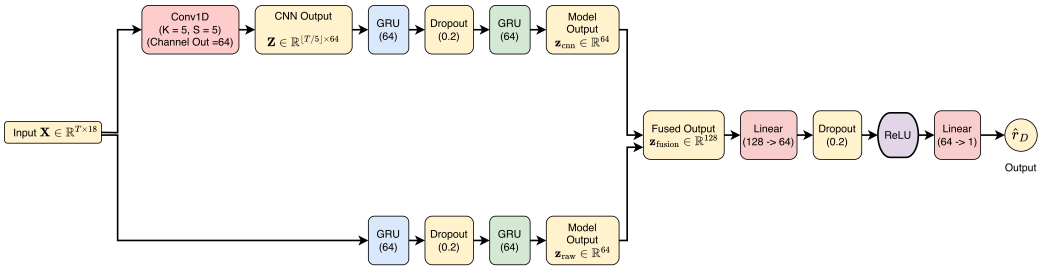


Figure 9. Overall structure of GRUDualBranchFusion. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

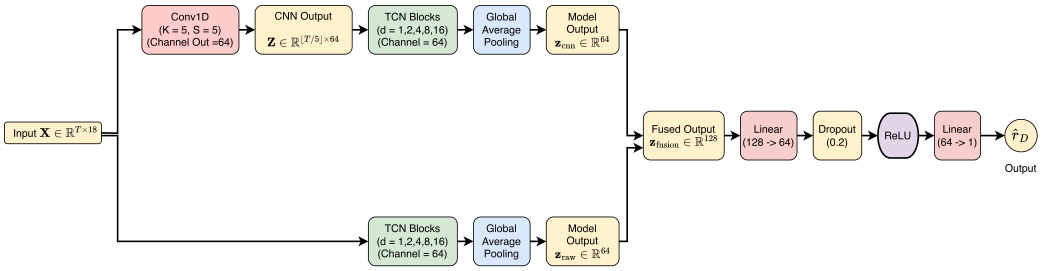


Figure 10. Overall structure of TCNDualBranchFusion. The model takes as input the feature matrix $\mathbf{X} \in \mathbb{R}^{T\times 18}$ of day D, and outputs the predicted return $\hat{r}_D$ for day D.

3.4.4. Model Configuration Summary

The hyperparameter configurations of all nine models are summarised in Table 3.

Table 3. Model configurations.

Category	Model	Parameter	Value
Baseline	LSTM	Num Layers	2
		Bidirectional	False
		Hidden Size	64
		Dropout Rate	0.2
	GRU	Num Layers	2
		Bidirectional	False
		Hidden Size	64
		Dropout Rate	0.2
	TCN	Num Blocks	5
		Dilation Factors	$\{1, 2, 4, 8, 16\}$
		Kernel Size	3
		Channel Size	64
		Dropout Rate	0.2
CNN Hybrid	CNN-LSTM	CNN Kernel Size	5
		CNN Stride	5
		CNN Output Channels	64
		LSTM Num Layers	2
		LSTM Hidden Size	64
		Dropout Rate	0.2
	CNN-GRU	CNN Kernel Size	5
		CNN Stride	5
		CNN Output Channels	64
		GRU Num Layers	2
		GRU Hidden Size	64
		Dropout Rate	0.2
	CNN-TCN	CNN Kernel Size	5
		CNN Stride	5
		CNN Output Channels	64
		TCN Num Blocks	5
		TCN Dilation Factors	$\{1, 2, 4, 8, 16\}$
		TCN Kernel Size	3
		TCN Channel Size	64
		Dropout Rate	0.2

Table 3. *Cont.*

Category	Model	Parameter	Value
Dual-Branch Fusion	LSTMDualBranchFusion	Raw Branch	LSTM
		CNN Branch	CNN-LSTM
		Fusion Hidden Size	64
		Dropout Rate	0.2
	GRUDualBranchFusion	Raw Branch	GRU
		CNN Branch	CNN-GRU
		Fusion Hidden Size	64
		Dropout Rate	0.2
	TCNDualBranchFusion	Raw Branch	TCN
		CNN Branch	CNN-TCN
		Fusion Hidden Size	64
		Dropout Rate	0.2

3.5. Experimental Setup

3.5.1. Training Configuration

Each of the nine model architectures is trained across five window sizes $W \in \{1, 2, 3, 4, 5\}$ (corresponding to input lengths $T \in \{240, 480, 720, 960, 1200\}$), yielding 45 distinct model configurations in total. Each configuration is independently trained and evaluated 20 times with different random seeds, resulting in $45 \times 20 = 900$ training runs in total.

All models are trained using the Adam optimiser (Kingma & Ba, 2014) with a learning rate of 1×10^{-3} and the Huber loss (Huber, 1992) (Smooth L1 Loss) as the training objective, defined as:

$$\mathcal{L}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| < 1 \\ |y - \hat{y}| - \frac{1}{2} & \text{otherwise} \end{cases} \tag{6}$$

where y and $\hat{y}$ denote the true and predicted log returns, respectively. The Huber loss is chosen over mean squared error for its robustness to outliers, which is particularly relevant in high-frequency financial data where return distributions are heavy-tailed (Cont, 2001).

Each model is trained for a maximum of 100 epochs with early stopping based on validation loss, with a patience of 10 epochs. The model checkpoint with the lowest validation loss is used for test set evaluation. Running each configuration 20 times with different random seeds accounts for the sensitivity of deep learning models to random initialisation (Henderson et al., 2018), and the mean and standard deviation of test set performance are reported across the 20 runs.

3.5.2. Evaluation Metrics

Model performance is evaluated on the test set using four metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Directional Accuracy (DA), and Information Coefficient (IC).

MAE and RMSE measure the magnitude of prediction errors:

$$\text{MAE} = \frac{1}{N} \sum_{t=1}^{N} |y_t - \hat{y}_t| \tag{7}$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{t=1}^{N} (y_t - \hat{y}_t)^2} \tag{8}$$

DA measures the proportion of correctly predicted return directions, and is widely used as a practically relevant evaluation metric in financial forecasting studies (Dai et al., 2022; Luo & Tian, 2020):

$$\mathrm{DA} = \frac{1}{N}\sum_{t=1}^{N} \mathbf{1}[\mathrm{sign}(y_t) = \mathrm{sign}(\hat{y}_t)] \tag{9}$$

IC measures the Pearson correlation between predicted and realised returns, and is a standard metric in quantitative finance for evaluating the rank-ordering ability of a predictive model (Q. Li et al., 2024):

$$\mathrm{IC} = \frac{\sum_{t=1}^{N}(y_t - \bar{y})(\hat{y}_t - \bar{\hat{y}})}{\sqrt{\sum_{t=1}^{N}(y_t - \bar{y})^2 \sum_{t=1}^{N}(\hat{y}_t - \bar{\hat{y}})^2}} \tag{10}$$

Trading-based performance metrics such as Sharpe ratio, portfolio return, and drawdown are not included in this study, as translating model predictions into a trading strategy requires additional assumptions regarding execution costs, position sizing, and market impact that fall outside the scope of this architectural comparison.

3.5.3. Statistical Testing

To assess whether performance differences between models are statistically significant, paired two-sample t-tests are conducted on the 20 independent test scores for each metric and window size combination. This protocol directly follows Dietterich (1998), which is the standard methodological reference for paired statistical testing in comparative machine learning studies. Each of the 20 scores is a scalar summary statistic computed over the test set from an independent training run; the independence of these runs is guaranteed by the different random initialisations. Using paired tests is appropriate here because both models in each comparison are trained and evaluated on the same dataset under the same conditions, with only the random seed differing across runs.

Since MAE and RMSE are lower-is-better metrics while DA and IC are higher-is-better metrics, the hypotheses are formulated accordingly. For a model pair (A, B) where model A is hypothesised to outperform model B:

- **MAE:** $H_0 : \mu^A_{\mathrm{MAE}} = \mu^B_{\mathrm{MAE}}$ vs. $H_1 : \mu^A_{\mathrm{MAE}} < \mu^B_{\mathrm{MAE}}$
- **RMSE:** $H_0 : \mu^A_{\mathrm{RMSE}} = \mu^B_{\mathrm{RMSE}}$ vs. $H_1 : \mu^A_{\mathrm{RMSE}} < \mu^B_{\mathrm{RMSE}}$
- **DA:** $H_0 : \mu^A_{\mathrm{DA}} = \mu^B_{\mathrm{DA}}$ vs. $H_1 : \mu^A_{\mathrm{DA}} > \mu^B_{\mathrm{DA}}$
- **IC:** $H_0 : \mu^A_{\mathrm{IC}} = \mu^B_{\mathrm{IC}}$ vs. $H_1 : \mu^A_{\mathrm{IC}} > \mu^B_{\mathrm{IC}}$

Three sets of pairwise comparisons are conducted:

1. **CNN hybrid vs. baseline** (CNN-LSTM vs. LSTM, CNN-GRU vs. GRU, CNN-TCN vs. TCN): evaluating whether CNN front-end feature extraction improves over direct sequence modelling.
2. **Dual-branch fusion vs. baseline** (LSTMDualBranchFusion vs. LSTM, GRUDualBranchFusion vs. GRU, TCNDualBranchFusion vs. TCN): evaluating whether the dual-branch architecture improves over the raw sequence baseline.
3. **Dual-branch fusion vs. CNN hybrid** (LSTMDualBranchFusion vs. CNN-LSTM, GRUDualBranchFusion vs. CNN-GRU, TCNDualBranchFusion vs. CNN-TCN): evaluating whether retaining the raw branch alongside the CNN branch provides additional gains.

Each comparison is conducted across 4 evaluation metrics and 5 window sizes, yielding $3 \times 3 \times 4 \times 5 = 180$ pairwise t-tests in total. A result is considered statistically significant at $\alpha = 0.05$.

4. Results

Table 4 reports the mean and standard deviation of all four metrics across 20 independent runs for each model and window size. Table 5 reports the paired t-test p-values for all pairwise comparisons. Several patterns emerge from the results.

Table 4. Model performance: mean (std) across 20 independent runs.

Metric	W	LSTM	GRU	TCN	CNN-LSTM	CNN-GRU	CNN-TCN	LSTM-Dual	GRU-Dual	TCN-Dual
MAE	1	0.0076 (0.0004)	0.0076 (0.0004)	0.0083 (0.0010)	0.0075 (0.0002)	0.0076 (0.0004)	0.0084 (0.0008)	**0.0074** (0.0002)	0.0076 (0.0007)	0.0076 (0.0004)
	2	0.0077 (0.0005)	0.0077 (0.0006)	0.0079 (0.0007)	0.0075 (0.0002)	0.0076 (0.0006)	0.0081 (0.0007)	**0.0074** (0.0001)	**0.0074** (0.0002)	0.0077 (0.0003)
	3	0.0074 (0.0002)	0.0077 (0.0004)	0.0079 (0.0008)	0.0075 (0.0002)	0.0078 (0.0004)	0.0080 (0.0008)	**0.0074** (0.0002)	0.0075 (0.0001)	0.0079 (0.0012)
	4	**0.0074** (0.0002)	0.0075 (0.0004)	0.0080 (0.0008)	0.0077 (0.0004)	0.0076 (0.0004)	0.0078 (0.0005)	0.0075 (0.0002)	0.0075 (0.0005)	0.0075 (0.0003)
	5	0.0075 (0.0002)	0.0076 (0.0003)	0.0081 (0.0010)	0.0076 (0.0003)	0.0078 (0.0008)	0.0077 (0.0004)	**0.0073** (0.0002)	0.0074 (0.0002)	0.0075 (0.0003)
RMSE	1	0.0104 (0.0004)	0.0103 (0.0003)	0.0110 (0.0010)	0.0102 (0.0002)	0.0104 (0.0003)	0.0111 (0.0008)	**0.0101** (0.0001)	0.0103 (0.0007)	0.0103 (0.0004)
	2	0.0104 (0.0004)	0.0104 (0.0005)	0.0107 (0.0006)	0.0103 (0.0001)	0.0104 (0.0005)	0.0107 (0.0006)	**0.0101** (0.0001)	0.0102 (0.0001)	0.0104 (0.0003)
	3	0.0102 (0.0002)	0.0104 (0.0004)	0.0107 (0.0007)	0.0103 (0.0002)	0.0106 (0.0003)	0.0108 (0.0008)	**0.0101** (0.0001)	0.0102 (0.0001)	0.0106 (0.0012)
	4	**0.0102** (0.0002)	0.0103 (0.0003)	0.0108 (0.0008)	0.0104 (0.0004)	0.0105 (0.0003)	0.0105 (0.0004)	**0.0102** (0.0002)	0.0103 (0.0004)	0.0103 (0.0002)
	5	0.0102 (0.0002)	0.0103 (0.0003)	0.0108 (0.0009)	0.0103 (0.0002)	0.0106 (0.0008)	0.0105 (0.0004)	**0.0101** (0.0001)	0.0102 (0.0002)	0.0103 (0.0003)
DA	1	0.4444 (0.0707)	0.4528 (0.0883)	0.4389 (0.0668)	0.4575 (0.0746)	0.4810 (0.0557)	0.4619 (0.0466)	0.4726 (0.0975)	**0.5008** (0.0978)	0.4762 (0.0918)
	2	0.4218 (0.0495)	0.4448 (0.0822)	0.4738 (0.0907)	0.4444 (0.0703)	**0.5067** (0.0804)	0.4373 (0.0675)	0.4794 (0.0900)	0.4492 (0.0750)	0.4246 (0.0874)
	3	0.4639 (0.0695)	0.4353 (0.0660)	**0.4877** (0.0972)	0.4540 (0.0719)	0.4536 (0.0710)	0.4595 (0.0696)	0.4266 (0.0775)	0.4206 (0.0459)	0.4679 (0.0850)
	4	0.4683 (0.0674)	0.4710 (0.0941)	0.4560 (0.0807)	0.4333 (0.0673)	0.4758 (0.0761)	0.4623 (0.0584)	0.4377 (0.0839)	0.4631 (0.0887)	**0.4794** (0.1012)
	5	0.4429 (0.0652)	0.4460 (0.0610)	0.4226 (0.0715)	0.4385 (0.0843)	0.4770 (0.0619)	0.4810 (0.0860)	**0.4833** (0.0852)	0.4679 (0.0953)	0.4290 (0.0821)
IC	1	0.1031 (0.0731)	0.0745 (0.0937)	−0.0211 (0.1358)	0.0187 (0.0872)	0.0284 (0.1145)	0.0680 (0.1022)	**0.1181** (0.0965)	0.0799 (0.1284)	0.0426 (0.1176)
	2	**0.1345** (0.0491)	0.0697 (0.0841)	−0.0138 (0.0907)	0.0205 (0.1208)	0.0246 (0.0861)	0.0416 (0.0819)	0.1083 (0.1052)	0.0907 (0.0949)	−0.0167 (0.0700)
	3	0.1278 (0.0769)	0.0877 (0.0813)	−0.0044 (0.0921)	−0.0088 (0.1016)	0.0172 (0.0906)	−0.0282 (0.1017)	**0.1310** (0.1097)	0.1014 (0.0799)	0.0082 (0.0924)
	4	0.0845 (0.1104)	0.0784 (0.0832)	−0.0095 (0.1125)	0.0236 (0.0965)	−0.0151 (0.1013)	0.0030 (0.0616)	**0.1129** (0.0850)	0.0640 (0.1191)	0.0017 (0.0964)
	5	**0.1071** (0.0773)	0.0997 (0.0829)	0.0055 (0.1127)	0.0365 (0.0986)	0.0089 (0.1149)	−0.0111 (0.0682)	0.0813 (0.1007)	0.0883 (0.0806)	−0.0277 (0.0793)

Note: W denotes window size in trading days. Values reported as mean (std) across 20 independent runs. **Bold** indicates the best mean performance per row; statistical significance of pairwise differences should be assessed from Table 5.

Table 5. Paired t-test p-values.

Metric	W	CNN-LSTM over LSTM	CNN-GRU over GRU	CNN-TCN over TCN	LSTM-Dual over LSTM	GRU-Dual over GRU	TCN-Dual over TCN	LSTM-Dual over CNN-LSTM	GRU-Dual over CNN-GRU	TCN-Dual over CNN-TCN
MAE	1	0.0917	0.6206	0.6735	**0.0095**	0.4023	**0.0068**	**0.0144**	0.3372	**0.0005**
	2	0.0658	0.3502	0.7681	**0.0016**	**0.0352**	0.0969	**0.0126**	0.1299	**0.0143**
	3	0.8579	0.8770	0.6929	0.5506	0.0580	0.4787	0.0956	**0.0007**	0.3375
	4	0.9863	0.7865	0.2166	0.7190	0.6051	**0.0103**	**0.0351**	0.2687	**0.0209**
	5	0.8983	0.7935	0.0521	**0.0176**	**0.0227**	**0.0086**	**0.0026**	**0.0362**	0.0861
RMSE	1	0.1379	0.7239	0.6497	**0.0234**	0.4419	**0.0045**	**0.0065**	0.3137	**0.0005**
	2	0.1375	0.4373	0.6227	**0.0046**	**0.0302**	0.0565	**0.0012**	0.0635	**0.0202**
	3	0.9789	0.9475	0.6170	0.4089	**0.0265**	0.4313	**0.0042**	**0.0001**	0.3564
	4	0.9831	0.9294	0.1059	0.5114	0.5344	**0.0093**	**0.0155**	**0.0463**	**0.0133**
	5	0.9679	0.9113	0.0723	**0.0270**	**0.0320**	**0.0107**	**0.0007**	**0.0124**	0.0705
DA	1	0.3113	0.0914	0.0819	0.1327	0.0744	0.0784	0.2689	0.2500	0.2637
	2	0.1196	**0.0190**	0.9156	**0.0053**	0.4336	0.9429	0.1002	0.9841	0.6778
	3	0.6564	0.2343	0.8385	0.9369	0.7592	0.7727	0.8842	0.9504	0.3734
	4	0.9614	0.4383	0.3426	0.8729	0.6032	0.1742	0.4265	0.6868	0.2547
	5	0.5705	0.0570	**0.0264**	0.0673	0.2186	0.3971	0.0691	0.6576	0.9871
IC	1	0.9982	0.8862	**0.0155**	0.3186	0.4429	0.0572	**0.0010**	0.0719	0.7765
	2	0.9997	0.9220	**0.0403**	0.9087	0.2344	0.5578	**0.0036**	**0.0323**	0.9866
	3	0.9997	0.9868	0.7871	0.4542	0.3263	0.3413	**0.0001**	**0.0061**	0.1206
	4	0.9527	0.9975	0.3354	0.1935	0.6897	0.3123	**0.0056**	**0.0164**	0.5194
	5	0.9923	0.9910	0.7113	0.8728	0.6442	0.9151	0.0862	**0.0103**	0.7242

Note: p-values from one-sided paired t-tests across 20 independent runs. **Bold** indicates $p < 0.05$.

First, CNN hybrid models do not consistently outperform their raw baselines on MAE and RMSE. All three CNN hybrid models, CNN-LSTM, CNN-GRU, and CNN-TCN, show no significant error reductions across most window sizes. The dual-branch fusion models show more consistent gains: LSTMDualBranchFusion and TCNDualBranchFusion achieve significant MAE and RMSE reductions over their raw baselines at multiple window sizes, while GRUDualBranchFusion shows smaller improvements.

Second, DA is noisy across all models and window sizes, with most values between 0.44 and 0.50 and no architecture showing consistent directional improvement.

Third, on IC, LSTM and its dual-branch fusion variants achieve the highest values across all window sizes (approximately 0.07 to 0.13), while TCN-based models show near-zero or negative IC regardless of architecture variant. Adding a CNN front-end does not improve IC consistently for any model family. LSTMDualBranchFusion and GRUDualBranchFusion maintain IC levels close to or slightly above their raw baselines.

Finally, window size has little effect on the relative ordering of models across any metric, suggesting that the patterns above are robust to the choice of input horizon within the range of 1 to 5 trading days.

5. Discussion

5.1. RQ1: Does CNN Feature Extraction Help?

With identical back-end model configurations, the results show little evidence that adding a CNN front-end consistently improves predictive performance on standardised 1 min data.

For LSTM-based models, CNN-LSTM does not significantly outperform LSTM on MAE or RMSE at any window size, and its IC values are substantially lower across all window sizes, e.g., 0.0187 vs. 0.1031 at $W = 1$, and -0.0088 vs. 0.1278 at $W = 3$. This suggests that the CNN front-end may suppress useful information present in the raw sequence rather than improving feature extraction.

For GRU-based models, the pattern is similar. CNN-GRU shows no significant MAE or RMSE improvement over GRU across any window size, and its IC is consistently lower than the GRU baseline.

For TCN-based models, CNN-TCN achieves a statistically significant IC improvement over TCN at $W = 1$ ($p = 0.0155$) and $W = 2$ ($p = 0.0403$), but this does not extend to MAE, RMSE, or DA, and the advantage disappears at larger window sizes.

Overall, the raw LSTM and GRU baselines and their dual-branch fusion variants achieve the highest IC values in the experiment, and simply adding a CNN front-end does not improve on this. After standardisation, recurrent models appear capable of extracting useful patterns directly from the raw 1 min sequence without CNN preprocessing.

5.2. RQ2: Does Dual-Branch Fusion Help?

Compared to raw baselines, dual-branch fusion models show more consistent improvements than CNN hybrid models do. LSTMDualBranchFusion achieves significant MAE and RMSE reductions over LSTM at $W = 1$, $W = 2$, and $W = 5$. GRUDualBranchFusion achieves significant MAE and RMSE reductions over GRU at $W = 2$ and $W = 5$. TCNDualBranchFusion achieves significant MAE and RMSE reductions over TCN at $W = 1$, $W = 4$, and $W = 5$. On IC, improvements over the raw baselines are limited, which is expected given that the raw LSTM and GRU baselines and their dual-branch fusion variants already achieve the highest IC values in the experiment.

When compared against CNN hybrid models, dual-branch fusion models outperform their counterparts on MAE and RMSE across most window sizes. LSTMDualBranchFusion achieves significant MAE and RMSE reductions over CNN-LSTM at $W = 1$, $W = 2$, $W = 4$,

and $W = 5$. GRUDualBranchFusion achieves significant MAE and RMSE reductions over CNN-GRU at $W = 3$ and $W = 5$. TCNDualBranchFusion achieves significant MAE and RMSE reductions over CNN-TCN at $W = 1$ and $W = 2$. On IC, LSTMDualBranchFusion outperforms CNN-LSTM at $W = 1$, $W = 2$, $W = 3$, and $W = 4$, and GRUDualBranchFusion outperforms CNN-GRU at $W = 2$, $W = 3$, $W = 4$, and $W = 5$.

These results suggest that the raw sequence branch carries predictive information that the CNN front-end discards, and that retaining both branches is beneficial.

5.3. Interpretation of Findings

The results of this study yield several noteworthy insights.

First, after standardisation, adding a CNN front-end does not improve performance and in many cases reduces IC relative to the raw baseline. This suggests that standardised 1 min sequences already provide a sufficient input representation for recurrent models, and that the CNN front-end may discard predictive information in this setting (Livieris et al., 2020; Z. Zhang et al., 2019).

Second, the dual-branch architecture consistently outperforms both the raw baseline and the CNN hybrid on MAE, RMSE, and IC for LSTM- and GRU-based models across most window sizes. The raw sequence and the CNN-compressed sequence appear to carry complementary information, and retaining both branches is beneficial.

Third, TCN-based models produce near-zero or negative IC values regardless of architecture variant or window size (Dai et al., 2022; C.-X. Zhang et al., 2022). This pattern is not resolved by CNN preprocessing or dual-branch fusion, suggesting a possible limitation of dilated convolutional architectures for return rank-ordering on this dataset and sample period. Whether this reflects a fundamental architectural mismatch or a tuning artefact specific to this setting remains an open question for future work.

Finally, overall predictive performance is modest: LSTM and GRU baselines achieve IC values of 0.07–0.13 and DA values of 0.42–0.51, consistent with the known difficulty of high-frequency return prediction from price-based features alone (Ait-Sahalia et al., 2005; Cont, 2001). It is important to note that the architectural conclusions of this study concern relative differences between models rather than the absolute level of predictive performance; the finding that dual-branch fusion consistently outperforms the pure CNN hybrid, for instance, holds regardless of the overall performance level. These patterns hold across all five window sizes examined.

All findings in this study are restricted to the CSI 300 index over the sample period of January 2015 to December 2025. Generalisation to other instruments, markets, or time periods remains to be examined, and extension along these dimensions is recommended as an important direction for future work. Rolling window evaluation across multiple test periods would provide additional evidence of temporal robustness, and mapping model predictions onto real trading portfolios, for example, by reporting the Sharpe ratio or drawdown, would provide evidence of practical utility beyond the statistical metrics reported here.

6. Conclusions

This study investigates whether CNN-based front-end feature extraction improves the predictive performance of deep learning models applied to 1 min intraday CSI 300 index data (Song et al., 2024). Three baseline sequence models, LSTM, GRU, and TCN, are compared against their CNN hybrid counterparts and dual-branch fusion variants across five input window sizes and four evaluation metrics, with statistical significance assessed via paired t-tests across 20 independent training runs (Dietterich, 1998). All comparisons use identical back-end model configurations, so that any performance differences are

attributable solely to the presence or absence of the CNN front-end or the additional raw sequence branch.

The results yield three main conclusions. First, after standardisation, adding a CNN front-end does not consistently improve performance over the raw baseline, and in many cases reduces IC for LSTM- and GRU-based models. This indicates that standardised recurrent models appear capable of extracting useful patterns directly from the raw 1 min sequence without CNN preprocessing. Second, the dual-branch fusion architecture, which retains both the raw sequence branch and the CNN-compressed branch, consistently outperforms the pure CNN hybrid on MAE, RMSE, and IC for LSTM- and GRU-based models across most window sizes. This suggests that the raw sequence carries predictive information that the CNN front-end discards, and that retaining both representations is beneficial (Kim & Kim, 2019). Third, TCN-based models produce near-zero or negative IC values regardless of architecture variant, suggesting a possible limitation of dilated convolutional architectures for return rank-ordering on this dataset and sample period (Dai et al., 2022; C.-X. Zhang et al., 2022) that is not resolved by either CNN preprocessing or dual-branch fusion.

All findings should be interpreted within the scope of the CSI 300 index and the sample period of January 2015 to December 2025. Extension to other markets, instruments, or time periods requires further investigation.

A key limitation of this study is that all CNN hybrid models share the same back-end configuration as their raw baseline counterparts. This design ensures a fair comparison but may not be well-suited for the CNN hybrid architecture. Since the CNN front-end reduces the sequence length by a factor of five, a simpler or shallower back-end may be more appropriate for the compressed input. Future work should explore back-end configurations specifically tuned for CNN-compressed sequences, which may recover some of the predictive performance lost in the current setup.

Further directions also include replicating the analysis on a broader range of financial instruments and markets, incorporating richer input representations such as limit order book data (Z. Zhang et al., 2019) or sentiment signals, investigating alternative CNN front-end designs such as multi-scale or attention-augmented encoders, conducting rolling window evaluation across multiple test periods to assess temporal robustness, and mapping model predictions onto real trading portfolios to evaluate practical utility.

Author Contributions: Conceptualisation, F.Z. and H.W.; methodology, F.Z. and H.W.; software, F.Z.; validation, F.Z. and H.W.; formal analysis, F.Z.; investigation, F.Z.; data curation, F.Z.; writing—original draft preparation, F.Z. and H.W.; writing—review and editing, F.Z. and H.W.; visualisation, F.Z.; supervision, H.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The CSI 300 1 min intraday data used in this study were obtained through a paid subscription to the BigQuant platform (https://bigquant.com). Access to the data is subject to the platform's terms of service and is not freely redistributable. Requests for further information regarding data access should be directed to the corresponding author. The full model implementation code is available at https://github.com/fan5211v/Paper_CNN_HF (accessed on 17 May 2026). Any researcher who is able to obtain equivalent 1 min intraday data from any compatible data provider and organise it according to the repository instructions will be able to reproduce the experimental results. The repository also includes guidance on adapting the pipeline to alternative input frequencies, such as 5 min data.

Acknowledgments: The authors thank the anonymous reviewers for their constructive comments. During the preparation of this manuscript, the authors used Claude Sonnet 4.5 (Anthropic, claude.ai) for the purposes of grammar checking and proofreading. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CNN	Convolutional Neural Network
GRU	Gated Recurrent Unit
IC	Information Coefficient
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
OHLCV	Open, High, Low, Close, Volume
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
TCN	Temporal Convolutional Network
DA	Directional Accuracy

References

Admati, A. R., & Pfleiderer, P. (1988). A theory of intraday patterns: Volume and price variability. *The Review of Financial Studies*, *1*(1), 3–40. [CrossRef]

Ait-Sahalia, Y., Mykland, P. A., & Zhang, L. (2005). How often to sample a continuous-time process in the presence of market microstructure noise. *The Review of Financial Studies*, *18*(2), 351–416. [CrossRef]

Aït-Sahalia, Y., & Yu, J. (2008). *High frequency market microstructure noise estimates and liquidity measures*: *Technical report*. National Bureau of Economic Research.

Andersen, T. G., Bollerslev, T., & Meddahi, N. (2011). Realized volatility forecasting and market microstructure noise. *Journal of Econometrics*, *160*(1), 220–234. [CrossRef]

Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv*, arXiv:1803.01271. [CrossRef]

Bandi, F. M., & Russell, J. R. (2006). Separating microstructure noise from volatility. *Journal of Financial Economics*, *79*(3), 655–692. [CrossRef]

Campbell, J. Y., Lo, A. W., MacKinlay, A. C., & Whitelaw, R. F. (1998). The econometrics of financial markets. *Macroeconomic Dynamics*, *2*(4), 559–562. [CrossRef]

Cerqueira, V., Torgo, L., & Mozetič, I. (2020). Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning*, *109*(11), 1997–2028. [CrossRef]

Cho, K., Van Merriënboer, B., Gulçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014, October 25–29). *Learning phrase representations using RNN encoder–decoder for statistical machine translation*. 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 1724–1734), Doha, Qatar.

Cont, R. (2001). Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, *1*(2), 223. [CrossRef]

Dai, W., An, Y., & Long, W. (2022). Price change prediction of ultra high frequency financial data based on temporal convolutional network. *Procedia Computer Science*, *199*, 1177–1183. [CrossRef]

Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, *10*(7), 1895–1923. [CrossRef] [PubMed]

Gunduz, H., Yaslan, Y., & Cataltepe, Z. (2017). Intraday prediction of Borsa Istanbul using convolutional neural networks and feature correlations. *Knowledge-Based Systems*, *137*, 138–148. [CrossRef]

Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., & Meger, D. (2018, February 2–7). *Deep reinforcement learning that matters*. AAAI Conference on Artificial Intelligence (Vol. 32), New Orleans, LA, USA.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780. [CrossRef] [PubMed]

Hoseinzade, E., & Haratizadeh, S. (2019). CNNpred: CNN-based stock market prediction using a diverse set of variables. *Expert Systems with Applications*, *129*, 273–285. [CrossRef]

Huber, P. J. (1992). Robust estimation of a location parameter. In *Breakthroughs in statistics: Methodology and distribution* (pp. 492–518). Springer.

Kim, T., & Kim, H. Y. (2019). Forecasting stock prices with a feature fusion LSTM-CNN model using different representations of the same data. *PLoS ONE*, *14*(2), e0212320. [CrossRef]

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv*, arXiv:1412.6980.

Li, C., & Qian, G. (2022). Stock price prediction using a frequency decomposition based GRU transformer neural network. *Applied Sciences*, *13*(1), 222. [CrossRef]

Li, Q., Kamaruddin, N., Yuhaniz, S. S., & Al-Jaifi, H. A. A. (2024). Forecasting stock prices changes using long-short term memory neural network with symbolic genetic programming. *Scientific Reports*, *14*(1), 422. [CrossRef] [PubMed]

Liu, L. Y., Patton, A. J., & Sheppard, K. (2015). Does anything beat 5 min RV? A comparison of realized measures across multiple asset classes. *Journal of Econometrics*, *187*(1), 293–311. [CrossRef]

Livieris, I. E., Pintelas, E., & Pintelas, P. (2020). A CNN–LSTM model for gold price time-series forecasting. *Neural Computing and Applications*, *32*(23), 17351–17360. [CrossRef]

Luo, S., & Tian, C. (2020). Financial high-frequency time series forecasting based on sub-step grid search long short-term memory network. *IEEE Access*, *8*, 203183–203189. [CrossRef]

Passalis, N., Tefas, A., Kanniainen, J., Gabbouj, M., & Iosifidis, A. (2019). Deep adaptive input normalization for time series forecasting. *IEEE Transactions on Neural Networks and Learning Systems*, *31*(9), 3760–3765. [CrossRef] [PubMed]

Singh, P., & Raman, B. (2026). Dual-branch spectral-trend attention network with gated flux–momentum decomposition for multiscale financial time-series forecasting. *Journal of Forecasting*. [CrossRef]

Song, Y., Cai, C., Ma, D., & Li, C. (2024). Modelling and forecasting high-frequency data with jumps based on a hybrid nonparametric regression and LSTM model. *Expert Systems with Applications*, *237*, 121527. [CrossRef]

Stitsyuk, A., & Choi, J. (2025, February 25–March 4). *xpatch: Dual-stream time series forecasting with exponential seasonal-trend decomposition*. AAAI Conference on Artificial Intelligence (Vol. 39, pp. 20601–20609), Philadelphia, PA, USA.

Sun, G., & Li, Y. (2025). Intraday and Post-Market investor sentiment for stock price prediction: A deep learning framework with explainability and quantitative trading strategy. *Systems*, *13*(5), 390. [CrossRef]

Zhang, C.-X., Li, J., Huang, X.-F., Zhang, J.-S., & Huang, H.-C. (2022). Forecasting stock volatility and value-at-risk based on temporal convolutional networks. *Expert Systems with Applications*, *207*, 117951. [CrossRef]

Zhang, Z., Zohren, S., & Roberts, S. (2019). Deeplob: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, *67*(11), 3001–3012. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.